

SQL — Fiche de révision

Concours ANP — Cadre Data Scientist

1 Requête de base : SELECT, WHERE

```
-- Sélectionner des colonnes précises
SELECT nom, tirant_eau
FROM navires;

-- Sélectionner toutes les colonnes
SELECT * FROM navires;

-- Filtrer les lignes avec WHERE
SELECT nom, tirant_eau
FROM navires
WHERE tirant_eau > 14;

-- Combiner des conditions
SELECT * FROM navires
WHERE port = 'Casablanca' AND tirant_eau > 12;

-- Filtrer sur un ensemble de valeurs ou un motif
SELECT * FROM navires WHERE port IN ('Tanger', 'Agadir');
SELECT * FROM navires WHERE nom LIKE 'MSC%';
SELECT * FROM navires WHERE tirant_eau BETWEEN 10 AND 15;
SELECT * FROM navires WHERE date_arrivee IS NULL;
```

- SELECT choisit les colonnes à retourner ; FROM indique la table source.
- WHERE filtre les **lignes individuelles avant** tout regroupement — ne peut pas utiliser une fonction d'agrégation (COUNT, SUM...) directement.
- IS NULL / IS NOT NULL : seule façon correcte de tester une valeur NULL (= NULL ne fonctionne pas en SQL).

2 Regroupement : GROUP BY, HAVING

```
-- Nombre de navires par port
SELECT port, COUNT(*) AS nb_navires
FROM navires
GROUP BY port;

-- Tirant d'eau moyen par port et par type de navire
SELECT port, type_navire, AVG(tirant_eau) AS tirant_moyen
FROM navires
GROUP BY port, type_navire;

-- HAVING : filtrer APRES le regroupement, sur le resultat de l'agregation
SELECT port, COUNT(*) AS nb_navires
FROM navires
GROUP BY port
HAVING COUNT(*) > 10;
```

- **GROUP BY** regroupe les lignes ayant la même valeur dans la ou les colonnes indiquées, pour appliquer une fonction d'agrégation (COUNT, SUM, AVG, MIN, MAX) à chaque groupe.
- **Différence clé WHERE vs HAVING** : **WHERE** filtre les lignes **avant** le **GROUP BY** (ne connaît pas encore les agrégats); **HAVING** filtre les groupes **après** l'agrégation (peut utiliser COUNT(*), AVG(...), etc. dans sa condition).
- Toute colonne du **SELECT** qui n'est pas agrégée doit apparaître dans le **GROUP BY**.

3 Tri : ORDER BY

```
SELECT nom, tirant_eau FROM navires
ORDER BY tirant_eau DESC; -- décroissant

SELECT nom, tirant_eau FROM navires
ORDER BY port ASC, tirant_eau DESC; -- tri multi-colonnes

SELECT port, COUNT(*) AS nb_navires
FROM navires
GROUP BY port
ORDER BY nb_navires DESC
LIMIT 5; -- top 5 des ports
```

- **ASC** (croissant, par défaut) ou **DESC** (décroissant).
- **ORDER BY** s'exécute en dernier dans la logique de la requête (après **WHERE**, **GROUP BY**, **HAVING**), souvent combiné à **LIMIT** pour un "top N".

4 Jointures : JOIN, INNER JOIN, LEFT JOIN

```
-- INNER JOIN : uniquement les lignes qui correspondent dans les deux tables
SELECT n.nom, n.tirant_eau, p.nom AS port_nom
FROM navires n
INNER JOIN ports p ON n.port_id = p.id;

-- LEFT JOIN : toutes les lignes de la table de gauche (navires),
-- meme sans correspondance dans ports (colonnes de droite a NULL)
SELECT n.nom, p.nom AS port_nom
FROM navires n
LEFT JOIN ports p ON n.port_id = p.id;

-- Jointure sur plusieurs tables
SELECT n.nom, p.nom AS port_nom, e.type_escale
FROM navires n
INNER JOIN ports p ON n.port_id = p.id
INNER JOIN escales e ON e.navire_id = n.id;
```

- **JOIN** seul est équivalent à **INNER JOIN** dans la plupart des SGBD.
- **INNER JOIN** : ne garde que les lignes où la condition **ON** trouve une correspondance dans les deux tables (intersection).
- **LEFT JOIN** (ou **LEFT OUTER JOIN**) : garde **toutes** les lignes de la table de gauche, avec **NULL** pour les colonnes de droite si aucune correspondance n'existe — utile pour repérer les lignes **sans** correspondance (**WHERE p.id IS NULL**).
- Toujours préfixer les colonnes ambiguës par l'alias de table (**n.nom** vs **p.nom**) dès qu'on joint plusieurs tables.

5 Ordre logique d'exécution d'une requête

Ordre	Clause
1	FROM / JOIN
2	WHERE
3	GROUP BY
4	HAVING
5	SELECT
6	ORDER BY
7	LIMIT

- Explique pourquoi `WHERE` ne peut pas utiliser un alias défini dans `SELECT`, et pourquoi `HAVING` peut utiliser un agrégat alors que `WHERE` non.

6 Questions types (révision active)

1. Quelle est la différence fondamentale entre `WHERE` et `HAVING` ?
2. Pourquoi `WHERE tirant_eau = NULL` ne fonctionne-t-il jamais ?
3. Quelle est la différence entre `INNER JOIN` et `LEFT JOIN` ?
4. Comment retrouver les navires qui n'ont **aucune** correspondance dans la table `ports` ?
5. Dans quel ordre logique une requête SQL avec `WHERE`, `GROUP BY`, `HAVING` et `ORDER BY` est-elle exécutée ?
6. Écrivez une requête donnant le top 3 des ports par nombre de navires.

7 Corrigé des questions types

Q1. Différence `WHERE` / `HAVING` ?

`WHERE` filtre les lignes individuelles **avant** tout regroupement — il ne connaît pas encore le résultat d'une agrégation et ne peut donc pas utiliser `COUNT(*)`, `AVG(...)`, etc. `HAVING` filtre les **groupes après** l'exécution du `GROUP BY` et peut donc conditionner sur le résultat d'une fonction d'agrégation.

Q2. Pourquoi `WHERE tirant_eau = NULL` ne fonctionne jamais ?

En SQL, `NULL` représente une valeur inconnue, et toute comparaison avec `NULL` (via `=`, `!=`, etc.) renvoie `NULL` (ni vrai ni faux), jamais `TRUE` — la ligne n'est donc jamais retenue. Il faut utiliser les opérateurs dédiés `IS NULL` ou `IS NOT NULL`.

Q3. Différence `INNER JOIN` / `LEFT JOIN` ?

`INNER JOIN` ne conserve que les lignes qui ont une correspondance dans les **deux** tables (intersection selon la condition `ON`). `LEFT JOIN` conserve **toutes** les lignes de la table de gauche, en complétant par `NULL` les colonnes de la table de droite quand aucune correspondance n'existe.

Q4. Navires sans correspondance dans `ports` ?

```
SELECT n.*
FROM navires n
LEFT JOIN ports p ON n.port_id = p.id
WHERE p.id IS NULL;
```

Le `LEFT JOIN` garde tous les navires, y compris ceux sans port correspondant (colonnes de `ports` à `NULL`) ; le `WHERE p.id IS NULL` isole ensuite uniquement ces lignes sans correspondance.

Q5. Ordre logique d'exécution ?

`FROM/JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`. Les tables sont d'abord assemblées et filtrées ligne par ligne, puis regroupées et filtrées par groupe, avant que les colonnes finales ne soient calculées et triées.

Q6. Top 3 des ports par nombre de navires :

```
SELECT port, COUNT(*) AS nb_navires
FROM navires
GROUP BY port
ORDER BY nb_navires DESC
LIMIT 3;
```

8 CASE WHEN

CASE WHEN crée une colonne conditionnelle calculée, l'équivalent SQL d'un if/elif/else.

```
SELECT
    nom,
    tirant_eau,
    CASE
        WHEN tirant_eau < 8 THEN 'petit navire'
        WHEN tirant_eau < 14 THEN 'navire moyen'
        ELSE 'grand navire'
    END AS categorie
FROM navires;

-- Utilisable aussi dans un GROUP BY / agregation
SELECT
    CASE WHEN tirant_eau >= 14 THEN 'grand' ELSE 'standard' END AS categorie,
    COUNT(*) AS nb_navires
FROM navires
GROUP BY
    CASE WHEN tirant_eau >= 14 THEN 'grand' ELSE 'standard' END;
```

- Les conditions WHEN sont évaluées **dans l'ordre** ; dès qu'une est vraie, les suivantes sont ignorées.
- ELSE est optionnel : sans lui, les lignes ne correspondant à aucun WHEN reçoivent NULL.
- Peut apparaître dans SELECT, WHERE, GROUP BY ou ORDER BY.

Exercice : écrivez une requête qui affiche, pour chaque navire, une colonne **alerte** valant 'a verifier' si tirant_eau est NULL, 'critique' si tirant_eau > 15, sinon 'ok'.

9 Sous-requêtes

Une **sous-requête** (*subquery*) est une requête SELECT imbriquée dans une autre requête. Elle peut apparaître dans le WHERE, le FROM ou le SELECT.

```
-- Sous-requete dans le WHERE : navires plus grands que la moyenne
SELECT nom, tirant_eau
FROM navires
WHERE tirant_eau > (SELECT AVG(tirant_eau) FROM navires);

-- Sous-requete avec IN : ports ayant deja reçu un porte-conteneurs
SELECT nom FROM ports
WHERE id IN (
    SELECT port_id FROM navires WHERE type_navire = 'porte-conteneurs'
);

-- Sous-requete correelee : navires dont le tirant d'eau
-- depasse la moyenne de LEUR PROPRE port (reference a la requete externe)
SELECT n.nom, n.port_id, n.tirant_eau
FROM navires n
WHERE n.tirant_eau > (
    SELECT AVG(n2.tirant_eau)
    FROM navires n2
    WHERE n2.port_id = n.port_id
```

```
);

-- Sous-requete dans le FROM (table derivee)
SELECT port_id, moyenne_tirant
FROM (
    SELECT port_id, AVG(tirant_eau) AS moyenne_tirant
    FROM navires
    GROUP BY port_id
) AS stats_port
WHERE moyenne_tirant > 12;
```

- Sous-requête **non corrélée** : s'exécute une seule fois, indépendamment de la requête externe.
- Sous-requête **corrélée** : référence une colonne de la requête externe, donc réévaluée pour **chaque ligne** — plus coûteuse, mais nécessaire pour comparer une ligne à son propre groupe.
- Une sous-requête dans le **FROM** doit obligatoirement avoir un alias (ex. **AS stats_port**).

Exercice : écrivez une requête qui retourne les ports dont le nombre de navires est supérieur à la moyenne du nombre de navires par port (indice : sous-requête dans le **FROM** ou le **HAVING**).

10 CTE (Common Table Expression)

Une **CTE** (**WITH ... AS**) nomme une sous-requête temporaire réutilisable, pour rendre une requête complexe plus lisible qu'avec des sous-requêtes imbriquées.

```
WITH stats_port AS (
    SELECT port_id, AVG(tirant_eau) AS moyenne_tirant, COUNT(*) AS nb_navires
    FROM navires
    GROUP BY port_id
)
SELECT p.nom, sp.moyenne_tirant, sp.nb_navires
FROM stats_port sp
JOIN ports p ON p.id = sp.port_id
WHERE sp.nb_navires > 10;

-- Plusieurs CTE chainees
WITH escales_2024 AS (
    SELECT * FROM escales WHERE EXTRACT(YEAR FROM date_arrivee) = 2024
),
escales_par_port AS (
    SELECT port_id, COUNT(*) AS nb_escales
    FROM escales_2024
    GROUP BY port_id
)
SELECT p.nom, ep.nb_escales
FROM escales_par_port ep
JOIN ports p ON p.id = ep.port_id
ORDER BY ep.nb_escales DESC;

-- CTE recursive : ex. hierarchie ou serie generee
WITH RECURSIVE mois AS (
    SELECT 1 AS numero_mois
    UNION ALL
    SELECT numero_mois + 1 FROM mois WHERE numero_mois < 12
)
SELECT * FROM mois;
```

- Équivalente à une sous-requête dans le **FROM**, mais nommée **avant** la requête principale → plus lisible, surtout si réutilisée plusieurs fois.
- Plusieurs CTE peuvent être chaînées avec des virgules, chacune pouvant s'appuyer sur la précédente.

- `WITH RECURSIVE` permet de construire une hiérarchie ou une série (rare en entretien, mais bon à savoir reconnaître).

Exercice : réécrivez l'exercice de la section précédente (ports avec plus de navires que la moyenne) en utilisant une CTE au lieu d'une sous-requête imbriquée.

11 Window functions (fonctions de fenêtrage)

Une **window function** calcule une valeur sur un ensemble de lignes liées à la ligne courante (une "fenêtre"), **sans regrouper** les lignes comme le ferait `GROUP BY` — chaque ligne d'origine reste visible dans le résultat.

```
-- ROW_NUMBER : numérote les lignes dans chaque partition
SELECT
    nom, port_id, tirant_eau,
    ROW_NUMBER() OVER (PARTITION BY port_id ORDER BY tirant_eau DESC) AS rang
FROM navires;

-- RANK / DENSE_RANK : comme ROW_NUMBER mais gere les ex-aequo
SELECT
    nom, tirant_eau,
    RANK() OVER (ORDER BY tirant_eau DESC) AS rang,
    DENSE_RANK() OVER (ORDER BY tirant_eau DESC) AS rang_dense
FROM navires;

-- Agregat en fenetre : moyenne du port, affichee sur CHAQUE ligne
SELECT
    nom, port_id, tirant_eau,
    AVG(tirant_eau) OVER (PARTITION BY port_id) AS moyenne_port
FROM navires;

-- LAG / LEAD : valeur de la ligne precedente / suivante
SELECT
    nom, date_arrivee,
    LAG(date_arrivee) OVER (ORDER BY date_arrivee) AS arrivee_precedente,
    LEAD(date_arrivee) OVER (ORDER BY date_arrivee) AS arrivee_suivante
FROM escales;

-- Top 3 navires par port (ROW_NUMBER + CTE)
WITH classement AS (
    SELECT
        nom, port_id, tirant_eau,
        ROW_NUMBER() OVER (PARTITION BY port_id ORDER BY tirant_eau DESC) AS rang
    FROM navires
)
SELECT * FROM classement WHERE rang <= 3;
```

- `PARTITION BY` : découpe les lignes en groupes (comme `GROUP BY`), mais **sans fusionner** les lignes — chaque ligne garde son identité.
- `ORDER BY` dans le `OVER(...)` : définit l'ordre à l'intérieur de chaque partition (nécessaire pour `ROW_NUMBER`, `RANK`, `LAG/LEAD`).
- **ROW_NUMBER** : 1,2,3,4... toujours unique, même en cas d'égalité. **RANK** : ex-aequo reçoivent le même rang, mais saute des numéros après (1,2,2,4). **DENSE_RANK** : ex-aequo reçoivent le même rang, sans sauter de numéro (1,2,2,3).
- Impossible d'utiliser directement une window function dans un `WHERE` → il faut l'encapsuler dans une CTE ou une sous-requête (comme dans l'exemple "Top 3 par port").

Exercice : écrivez une requête qui affiche, pour chaque navire, son tirant d'eau et l'écart entre son tirant d'eau et la moyenne de son port (indice : `AVG(...) OVER (PARTITION BY ...)`).

12 Questions types (révision active) — partie avancée

1. Quelle est la différence entre une sous-requête corrélée et non corrélée ?
2. Pourquoi une CTE est-elle souvent préférée à une sous-requête imbriquée dans le FROM ?
3. Quelle est la différence entre ROW_NUMBER, RANK et DENSE_RANK ?
4. Que fait PARTITION BY dans une window function, et en quoi diffère-t-il de GROUP BY ?
5. Pourquoi ne peut-on pas filtrer directement sur une window function avec WHERE ?
6. Écrivez une requête donnant le top 3 des navires (par tirant d'eau) de chaque port.

13 Corrigé des questions types — partie avancée

Q1. Sous-requête corrélée vs non corrélée ?

Une sous-requête **non corrélée** est indépendante de la requête externe : elle s'exécute une seule fois et renvoie un résultat fixe (ex. la moyenne globale). Une sous-requête **corrélée** référence une colonne de la ligne courante de la requête externe (ex. `n2.port_id = n.port_id`), donc elle est réévaluée pour **chaque ligne** traitée — plus coûteuse en performance, mais indispensable pour comparer une ligne à son propre sous-groupe.

Q2. Pourquoi une CTE est-elle préférée à une sous-requête imbriquée ?

Une CTE nomme la sous-requête **avant** de l'utiliser (`WITH nom AS (...)`), ce qui rend la requête principale plus lisible et évite d'empiler des parenthèses imbriquées. Elle peut aussi être **réutilisée plusieurs fois** dans la même requête, et plusieurs CTE peuvent être chaînées pour découper un calcul complexe en étapes logiques successives.

Q3. Différence ROW_NUMBER / RANK / DENSE_RANK ?

Les trois numérotent les lignes selon un ordre donné, mais gèrent différemment les ex-aequo. ROW_NUMBER attribue toujours un numéro unique (1,2,3,4), même en cas d'égalité de valeurs. RANK donne le même rang aux ex-aequo mais **saute** les numéros suivants (1,2,2,4). DENSE_RANK donne le même rang aux ex-aequo **sans sauter** de numéro (1,2,2,3).

Q4. PARTITION BY vs GROUP BY ?

GROUP BY **fusionne** les lignes d'un même groupe en une seule ligne de résultat (une ligne par port, par exemple). PARTITION BY (dans une window function) découpe aussi les lignes en groupes logiques, mais **garde chaque ligne d'origine** dans le résultat, en ajoutant simplement une colonne calculée sur l'ensemble de sa partition (ex. la moyenne du port affichée sur chaque navire de ce port).

Q5. Pourquoi pas de WHERE direct sur une window function ?

Les window functions sont évaluées **après** le WHERE dans l'ordre logique d'exécution de la requête (après SELECT en réalité, quasiment en dernier avec ORDER BY) — au moment où WHERE s'exécute, leur résultat n'existe pas encore. Il faut donc encapsuler la window function dans une CTE ou une sous-requête, puis filtrer sur son résultat dans la requête englobante.

Q6. Top 3 navires par port :

```
WITH classement AS (  
    SELECT  
        nom, port_id, tirant_eau,  
        ROW_NUMBER() OVER (PARTITION BY port_id ORDER BY tirant_eau DESC) AS rang  
    FROM navires  
)  
SELECT nom, port_id, tirant_eau  
FROM classement  
WHERE rang <= 3;
```